

Synchronisation dans les processeurs multicœurs

César Fuguet

< cesar.fuguet@univ-grenoble-alpes.fr >

Chargé de Recherche, Inria, TIMA, Grenoble

Plan

- 1 Introduction
- 2 Opérations atomiques
 - Introduction
 - Implémentation
- Instructions atomiques
- 3 Quelques mécanismes de synchronisation
 - Verrous
 - Barrières ou rendez-vous
- 4 Exercices

Plan

1 Introduction

2 Opérations atomiques

- Introduction
- Implémentation

- Instructions atomiques

3 Quelques mécanismes de synchronisation

- Verrous
- Barrières ou rendez-vous

4 Exercices

Traitement multi-tâches

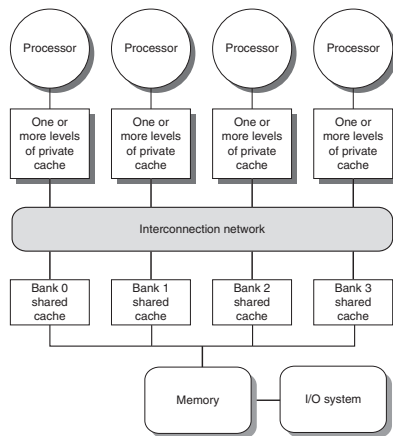
Les processeurs multicœurs permettent l'exécution parallèle de plusieurs tâches s'exécutant chacune sur un cœur de calcul.

Tâches coopératives

- Tâches appartenant au même processus (« parallel processing »).
- Dialogue entre tâches afin de résoudre un même problème.

Tâches concurrentes

- Tâches appartenant à différents processus (« multi-programming »).
- Dialogue entre les tâches afin de réserver (ou libérer) des ressources partagées du système (p.e. Ports d'entrée et sortie).



(source : Hennessy & Patterson (2012))

Exemple : Incrément d'un compteur de manière concurrentielle (I)

Tâche A

Tâche B

*compteur = 0

ancienne_valeur = *compteur

nouvelle_valeur = ancienne_valeur + 1

*compteur = nouvelle_valeur

*compteur = 1

ancienne_valeur = *compteur

nouvelle_valeur = ancienne_valeur + 1

*compteur = nouvelle_valeur

*compteur = 2

Exemple : Incrément d'un compteur de manière concurrentielle (II)

Tâche A

Tâche B

$*compteur = 0$

$ancienne_valeur = *compteur$

$nouvelle_valeur = ancienne_valeur + 1$

$ancienne_valeur = *compteur$

$nouvelle_valeur = ancienne_valeur + 1$

$*compteur = nouvelle_valeur$

$*compteur = 1$

$*compteur = nouvelle_valeur$

$*compteur = 1$

Ceci est une situation de compétition ou « race condition » !

Synchronisation

Définition

Ce sont des mécanismes permettant d'assurer que les opérations (parallèles), dans un programme parallèle, s'exécutent dans le « bon ordre ».

- La synchronisation permet d'établir un ordre séquentiel d'exécution afin de gérer les situations de compétition (« race conditions »),
- permet de fixer des points de rendez-vous entre tâches,
- permet d'assurer la complétude des transferts de données (« handshake »).

Implémentation

Généralement, ce sont des routines logicielles s'appuyant sur des mécanismes au niveau matériel (notamment les **opérations atomiques**).

Dans ce cours on introduira les concepts des mécanismes de synchronisation dans les architectures multicœurs à mémoire partagée.

Rappel

« Performance is now a programmer's burden. The La-Z-Boy programmer era of relying on hardware designers to make their programs go faster without lifting a finger is officially over »

(Hennessy & Patterson)

Plan

1 Introduction

2 Opérations atomiques

- Introduction
- Implémentation

• Instructions atomiques

3 Quelques mécanismes de synchronisation

- Verrous
- Barrières ou rendez-vous

4 Exercices

Atomicité

Opération atomique

C'est un ensemble insécable d'opérations plus basiques dans lequel toutes les opérations doivent se faire entièrement et sans interruptions. Les instructions « **read-modify-write (RMW)** » permettant de lire, modifier et écrire une adresse mémoire sont une classe d'opération atomique.

Exemple de l'incrément d'un compteur par deux tâches parallèles

Il faut exécuter de manière ininterrompue les opérations lecture, incrément et écriture.

Tâche A

Tâche B

*compteur = 0

ancienne_valeur = *compteur (**lecture**)

nouvelle_valeur = ancienne_valeur + 1 (**modification**)

*compteur = nouvelle_valeur (**écriture**)

Opération Atomique

*compteur = 1

ancienne_valeur = *compteur (**lecture**)

nouvelle_valeur = ancienne_valeur + 1 (**modification**)

*compteur = nouvelle_valeur (**écriture**)

Opération Atomique

*compteur = 2

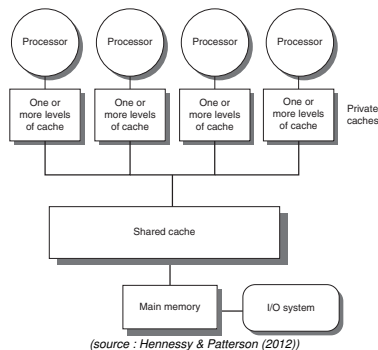
Comment implémenter les opérations atomiques ?

Multicœurs utilisant un bus pour l'interconnexion

- 1 Réserver le bus lors de la lecture en mémoire.
- 2 Modifier la donnée localement dans le cœur.
- 3 Relâcher le bus lors de l'écriture en mémoire.

Ce mécanisme ne passe pas à l'échelle.

Les processeurs avec un nombre de cœurs important (à partir de 8 en général), n'implémentent pas un bus car celui-ci limite fortement les performances.



Comment implémenter les opérations atomiques ?

Multicœurs utilisant un réseau d'interconnexion hiérarchique

Un mécanisme s'appuyant sur la réservation complète du réseau n'est plus envisageable car :

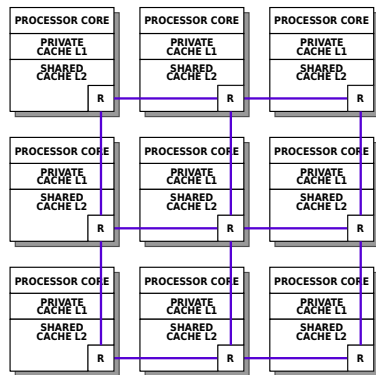
- Trop complexe et donc coûteux à implémenter.
- Trop pénalisant pour les performances et le passage à l'échelle.

Solution

Les processeurs modernes implémentent « nativement » des instructions atomiques.

L'implémentation peut consister en :

- Instructions spécifiques dans le jeu d'instructions des cœurs de processeur.
- Périphériques ou coprocesseurs proposant des primitives de synchronisation.



Instructions atomiques RISC-V

Load-Reserved/Store-Conditional

Mnémonique	Description	Syntaxe
LR	Load-Reserved	<code>lr.w \$rd, offset(\$rs1)</code>
SC	Store-Conditional	<code>sc.w \$rd, \$rs2, offset(\$rs1)</code>

Load-Reserved (LR)

Lit une valeur de la mémoire pour commencer une opération atomique de type RMW.

```
[ $rd ] = mem[ [ $rs1 ] + offset ]
```

Store-Conditional (SC)

Écrit une valeur dans la mémoire pour finir une opération atomique de type RMW.

Si `sc_atmique` alors

```
mem[ [ $rs1 ] + offset ] = [ $rs2 ]  
[ $rd ] = 0
```

Sinon

```
[ $rd ] = 1
```

Conditions

Un SC réussit (`sc_atmique`) quand :

- Un LR sur la même adresse l'a précédé.
- Pas d'interruptions entre le LR et le SC.
- L'adresse destination n'a pas été modifiée entre le LR et le SC.

Incrément d'un compteur avec LR/SC

Exercice

Écrire une routine en assembleur RISC-V permettant additionner une valeur à une adresse mémoire de manière atomique au moyen des instructions LR/SC.

```
int atomic_fetch_add(int *obj, int operand);
```

Faire en suite une exécution sur papier de la routine proposée avec deux tâches essayant d'incrémenter un compteur concurremment.

Échanger une valeur avec LR/SC

Exercice

Écrire une routine en assembleur RISC-V permettant d'échanger la valeur contenue à une adresse mémoire par une autre de manière atomique au moyen des instructions LR/SC.

```
int atomic_exchange(int *obj, int desired);
```

Instructions atomiques RISC-V (continuation)

RISC-V Atomic Memory Operations (AMO)

Mnémonique	Description	Syntaxe
AMOSWAP	Swap	<code>amoswap \$rd, \$rs2, 0(\$rs1)</code>
AMOADD	Integer add	<code>amoadd \$rd, \$rs2, 0(\$rs1)</code>
AMOAND	Logical and	<code>amoand \$rd, \$rs2, 0(\$rs1)</code>
AMOOOR	Logical or	<code>amoor \$rd, \$rs2, 0(\$rs1)</code>
AMOXOR	Logical xor	<code>amoxor \$rd, \$rs2, 0(\$rs1)</code>
AMOMAX	Unsigned integer maximum	<code>amomax \$rd, \$rs2, 0(\$rs1)</code>
AMOMIN	Unsigned integer minimum	<code>amomin \$rd, \$rs2, 0(\$rs1)</code>

AMO (Atomic Memory Operations)

Ce sont des instructions de type **lecture-modification-écriture**.

Paramètres :

- `$rd` : la valeur originale lue.
- `$rs1` : l'adresse mémoire à lire puis écrire.
- `$rs2` : la valeur à appliquer sur la valeur lue.

Les opérations sont sérialisées et réalisées directement par le contrôleur mémoire !

Relation entre les nouveaux standards C/C++ et les instructions atomiques RISC-V

```
#include <stdatomic.h>

void atomic_store(atomic_int *obj, int desired);
int atomic_load(atomic_int *obj);

int atomic_exchange(atomic_int *obj, int desired);

bool atomic_compare_exchange_strong(atomic_int *obj, int *expected, int desired);
bool atomic_compare_exchange_weak(atomic_int *obj, int *expected, int desired);

int atomic_fetch_add(atomic_int *obj, int operand);
int atomic_fetch_sub(atomic_int *obj, int operand);
int atomic_fetch_or(atomic_int *obj, int operand);
int atomic_fetch_xor(atomic_int *obj, int operand);
int atomic_fetch_and(atomic_int *obj, int operand);

bool atomic_flag_test_and_set(atomic_flag *obj);
void atomic_flag_clear(atomic_flag *obj);
```

Instruction atomiques RISC-V

Les AMO RISC-V permettent d'implémenter efficacement les opérations atomiques C11/C++11.

Plan

- 1 Introduction
- 2 Opérations atomiques
 - Introduction
 - Implémentation
- Instructions atomiques
- 3 Quelques mécanismes de synchronisation
 - Verrous
 - Barrières ou rendez-vous
- 4 Exercices

Verrous « locks »

Définition

C'est un mécanisme d'exclusion mutuelle permettant de prévenir des situations de compétition (« race condition ») sur l'accès à une ressource partagée (par exemple un port d'entrée-sortie ou un segment/case mémoire).

Implémentation

Le verrou peut être implémenté comme une variable dans la mémoire.

Essentiellement, deux méthodes sont associées à un verrou :

- **Prise (« lock_acquire »)** : change l'état du verrou de libre (valeur 0) à pris (valeur 1). Si le verrou est déjà pris, la ressource associée est en train d'être accédée par une autre tâche, donc l'appelant doit attendre.
- **Relâchement (« lock_release »)** : libère un verrou précédemment pris (écrire la valeur 0).

Verrous « locks » : exemple

```
/* debut de la section critique
 * prise du verrou
 */
lock_acquire(&list.lock);

/* recherche d'une cle dans une
 * liste chainee
 */
linked_list_find(&list, key);

/* fin de la section critique
 * relachement du verrou
 */
lock_release(&list.lock);
```

Raisonnement

Il faut protéger avec un verrou les opérations sur la liste chaînée afin d'assurer qu'il n'y a pas d'autres opérations en parallèle sur celle-ci.

Section critique

Section de code qui doit être exécutée par une seule tâche à la fois. Elle se situe entre la prise et le relâchement du verrou.

Verrous « locks » : prise de verrou

Prise de verrou

```
void lock_acquire(int *lock) {  
    /* attente */  
    while (*lock != 0);  
  
    /* prise */  
    *lock = 1;  
}
```

```
while:  
    lw    $t0,    ($a0)  
    bne   $t0,    $zero, while  
    addi  $t0,    $zero, 1  
    sw    $t0,    ($a0)
```

Verrous « locks » : prise de verrou

Prise de verrou

```
void lock_acquire(int *lock) {  
    /* attente */  
    while (*lock != 0);  
  
    /* prise */  
    *lock = 1;  
}  
  
while:  
    lw     $t0,    ($a0)  
    bne    $t0,    $zero, while  
    addi   $t0,    $zero, 1  
    sw     $t0,    ($a0)
```

Est-ce correcte cette implémentation ?

Verrous « locks » : prise de verrou

Prise de verrou

```
void lock_acquire(int *lock) {  
    /* attente */  
    while (*lock != 0);  
  
    /* prise */  
    *lock = 1;  
}  
  
while:  
    lw    $t0,    ($a0)  
    bne   $t0,    $zero, while  
    addi  $t0,    $zero, 1  
    sw    $t0,    ($a0)
```

Est-ce correcte cette implémentation ?

Non ! il y a une situation de compétition entre la lecture, la modification et l'écriture du verrou (« RMW »). Ces opérations doivent se faire de manière atomique.

Verrous « locks » : prise de verrou

Exercice

Proposez une ou plusieurs procédures en assembleur RISC-V pour prendre le verrou de manière atomique. Si vous avez trouvé plusieurs solutions, argumenter sur quelle des solutions est meilleure

```
void lock_acquire(int *lock);
```


Verrous « locks » : prise de verrou

Exercice

Proposez une ou plusieurs procédures en assembleur RISC-V pour prendre le verrou de manière atomique. Si vous avez trouvé plusieurs solutions, argumenter sur quelle des solutions est meilleure

```
void lock_acquire(int *lock);
```

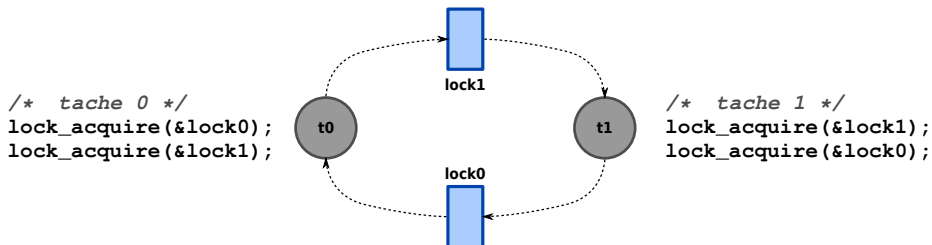
Exercice

Comment doivent les tâches relâcher le verrou ?

```
void lock_release(int *lock);
```

Verrous « locks » : interblocage

Situation d'interblocage ou « deadlock »



Solution

Si plusieurs verrous sont nécessaires, une stratégie de prise de verrou doit être définie afin d'éviter des interblocages. Le plus simple est de **prendre les verrous toujours dans le même ordre (verrous hiérarchiques)**.

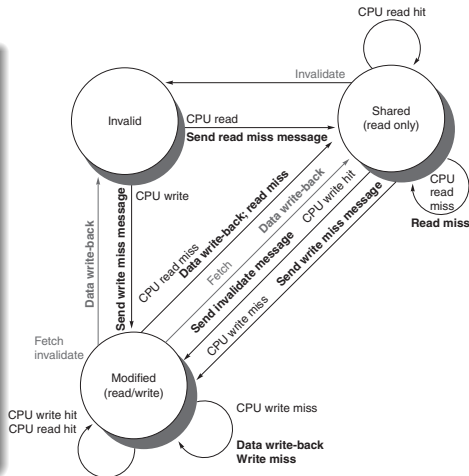
Solution de Dijkstra au problème du dîner des philosophes

Verrous « locks » : autres problèmes

Contention réseau et mémoire

La prise et relâchement d'un verrou peuvent engendrer beaucoup de transactions entre la mémoire et les caches (cohérence de caches).

- Prise de verrou :
 - $\Rightarrow \sum_{i=0}^{N-1} N - i$ lectures (miss)
 - $\Rightarrow \sum_{i=0}^{N-1} 1$ écritures = N
 - $\Rightarrow \sum_{i=0}^{N-1} N - i - 1$ invalidations de cache
 - $\Rightarrow \sum_{i=0}^{N-1} N - i - 1$ relectures (miss)
- Relâchement du verrou :
 - $\Rightarrow \sum_{i=0}^{N-1} N - i - 1$ invalidations de cache
- Nombre total de transactions :
 - $\Rightarrow 2 \cdot N^2 - 2 \cdot N \Rightarrow \mathbf{O(N^2)}$



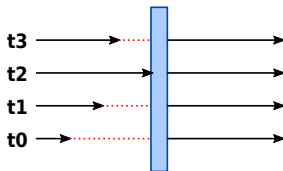
Famine

Certains algorithmes d'exclusion mutuelle (comme celui vu jusqu'ici) ne garantissent pas que toutes les tâches essayant de prendre le verrou réussiront.

Barrières ou rendez-vous (« barrier »)

Définition

Mécanisme permettant à plusieurs tâches de « se donner rendez-vous ». Toutes les tâches attendent à un point de synchronisation et n'avancent que lorsqu'elles sont toutes arrivées.



Implémentation

Consiste en un compteur partagé. Deux méthodes sont associées à une barrière :

- **Initialisation** : appelée par une seule tâche dans une région non-critique. Initialise la barrière avec le nombre de tâches à synchroniser.
- **Attente (« wait »)** : appelée lorsqu'une tâche atteint la barrière. Le compteur est décrémenté à chaque arrivée d'une tâche. Les tâches attendent tant que le compteur est supérieur à 0.

Barrières (« barrier ») : opérations

Opérations

```
typedef int barrier_t;

/* initialisation */
void barrier_init(barrier_t *barrier, int n);

/* attente */
void barrier_wait(barrier_t *barrier);
```

Exercice

Implémenter les deux méthodes `barrier_init` et `barrier_wait`.

Plan

- 1 Introduction
- 2 Opérations atomiques
 - Introduction
 - Implémentation
- Instructions atomiques
- 3 Quelques mécanismes de synchronisation
 - Verrous
 - Barrières ou rendez-vous
- 4 Exercices

Exercices

Dans les exercices qui suivent, nous allons implémenter des routines en relation avec les mécanismes de synchronisation introduits précédemment.

Pour les routines en assembleur, nous allons considérer que le processeur implémente le jeu d'instructions RISC-V (sauf mention particulière).

Exercice

Implémenter en assembleur la routine « compare et échange » (« compare & swap (CAS) » ou « compare & exchange ») :

```
int atomic_compare_exchange(int *obj, int *expected, int desired);
```

- La routine doit lire la valeur de l'adresse mémoire « **obj** » et comparer la valeur lue avec « ***expected** ». Si les deux valeurs sont égales, écrire à l'adresse « **obj** » la valeur « **desired** » et retourner « vrai » (valeur 1) ; sinon, ne rien modifier et retourner « faux » (valeur 0).
- En retournant, « ***expected** » doit contenir la valeur lue avant la modification.
- Les opérations de lecture et écriture sur « **obj** » doivent se faire de manière atomique car plusieurs tâches peuvent essayer de modifier cette adresse en parallèle.

Exercices

Exercice

Implémenter une routine permettant d'incrémenter de manière atomique une variable en utilisant « `atomic_compare_exchange` » :

```
int atomic_fetch_add(int *obj, int operand);
```

- La routine doit additionner la valeur « `operand` » à la valeur stockée dans l'adresse « `obj` ».
- La routine doit retourner la valeur lue avant l'incrément.

Exercices

Exercice

Implémenter les routines permettant de manipuler un verrou avec liste d'attente. **Cette méthode est celle actuellement implémentée dans le noyau Linux car elle garantit l'équité (« fairness ») et l'absence de famine, notamment dans les systèmes de type NUMA.**

```
struct ticket_lock_t {
    int curr_ticket;
    int last_ticket;
};

/* Cette routine permet de prendre un verrou. Pour le faire :
 * 1. la tache prend un ticket : elle lit "last_ticket" et
 *    l'incrémente pour préparer le ticket de la tache suivante,
 * 2. la tache attend son tour, c'est-à-dire, que la valeur de
 *    "curr_ticket" soit égale à celle de son ticket.
 */
void ticket_lock_acquire(ticket_lock_t *lock) ;

/* Cette routine permet à une tache de relâcher le verrou.
 */
void ticket_lock_release(ticket_lock_t *lock) ;
```